

Software Testing & Continuous Integration

Student: Daniela Lopez

Team: Group 1

Date: September 18, 2026

Phase 1 Test Coverage

Getting the environment running

Team's setup PR: <https://github.com/tylervu-unlv/Group-1/pull/37>

```
zombiegirl@pochacco: /mnt/c/Users/administrators1/Group-1/test_coverage_lab$ python3 -m pytest --cov=models --cov-report=term-missing
===== test session starts =====
platform linux -- Python 3.10.12, pytest-8.4.2, pluggy-1.6.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /mnt/c/Users/administrators1/Group-1/test_coverage_lab
configfile: pytest.ini
plugins: cov-7.1.0
collected 15 items

tests/test_account.py::test_account_role_assignment PASSED [ 6%]
tests/test_account.py::test_invalid_role_assignment PASSED [ 13%]
tests/test_account.py::test_invalid_email_format PASSED [ 20%]
tests/test_account.py::test_missing_name_raises_error PASSED [ 26%]
tests/test_account.py::test_missing_email_raises_error PASSED [ 33%]
tests/test_account.py::test_positive_deposit PASSED [ 40%]
tests/test_account.py::test_negative_or_zero_result[0] PASSED [ 46%]
tests/test_account.py::test_negative_or_zero_result[0.0] PASSED [ 53%]
tests/test_account.py::test_negative_or_zero_result[-100] PASSED [ 60%]
tests/test_account.py::test_negative_or_zero_result[-100.0] PASSED [ 66%]
tests/test_account.py::test_valid_withdrawal PASSED [ 73%]
tests/test_account.py::test_withdraw_insufficient_funds PASSED [ 80%]
tests/test_account.py::test_account_password_hashing PASSED [ 86%]
tests/test_account.py::test_account_deactivation_and_reactivation PASSED [ 93%]
tests/test_account.py::test_delete_account PASSED [100%]

===== tests coverage =====
----- coverage: platform linux, python 3.10.12-final-0 -----
Name                               Stmts  Miss  Cover   Missing
-----
models/__init__.py                   7      0  100%
models/account.py                   58      6   90%   30, 34, 60-62, 73
tests/__init__.py                     0      0  100%
tests/test_account.py               84      1   99%   184
TOTAL                             149      7   95%

===== 15 passed in 1.27s =====
```

Test #1 Invalid Email Input

Issue: <https://github.com/tylervu-unlv/Group-1/issues/40>

I picked this one up because `validate_email()` was sitting completely uncovered. My test constructs an Account with a garbage email like "not-an-email" and checks that `DataValidationError` actually gets raised — basically making sure the regex check does its job instead of quietly letting bad data through.

Pull request: <https://github.com/tylervu-unlv/Group-1/pull/42> **Merged**

Coverage before my test landed:

models\account.py	58	26	55%	30, 34, 47-49, 53-56, 60-62, 66-68, 72-76, 80, 84, 94, 98, 102-103
-------------------	----	----	-----	--

And after:

models\account.py	58	23	60%	30, 34, 53-56, 60-62, 66-68, 72-76, 80, 84, 94, 98, 102-103
-------------------	----	----	-----	---

Lines 47-49 — the regex check and its raise statement — are now actually exercised instead of just sitting there.

Test #2 Missing Required Fields

Issue: <https://github.com/tylervu-unlv/Group-1/issues/41>

This one had a small gotcha: SQLAlchemy doesn't validate anything at construction time, so just calling `Account()` with no arguments won't blow up on its own. I had to build the account and then explicitly call `validate_required_fields()` to actually trigger the check. I wrote two tests — one for a missing name, one for a missing email — so each failure mode gets its own clear signal.

Pull request: <https://github.com/tylervu-unlv/Group-1/pull/43> **Merged**

Coverage before:

models\account.py	58	23	60%	30, 34, 53-56, 60-62, 66-68, 72-76, 80, 84, 94, 98, 102-103
-------------------	----	----	-----	---

Coverage after:

models\account.py	58	19	67%	30, 34, 60-62, 66-68, 72-76, 80, 84, 94, 98, 102-103
-------------------	----	----	-----	--

Lines 53-56 — both required-field checks — are now covered.

Reviewing my teammates' work

I reviewed two of Bigolbean7804's PRs, both around deposit handling.

PR #45 positive deposit test: <https://github.com/tylervu-unlv/Group-1/pull/45>

“Nice, focused test! Clear assertion, and it correctly targets `deposit()`. One small thing before I approve: line 8 has an extra “ at the start of the docstring.”

PR #47 zero/negative deposit test: <https://github.com/tylervu-unlv/Group-1/pull/47>

“Nice! Just a couple of minor details worth fixing: the acceptance criteria text still says ‘verifies behavior when there is a positive deposit’ — should say zero/negative. Might be worth also asserting `account.balance == 0.0` after the exception, just to confirm the deposit didn't sneak through.”

What came back on my own PRs

- PR #42 — VegasBear approved it: “The test correctly exercises `validate_email()`. It verifies that a `ValidationError` is raised for an invalid email format. All tests pass. The PR fixes the correct issue.”
- PR #43 — clean approval, nothing requested.

Phase 2 Test-Driven Development

Setting up the counter service

Team's TDD setup PR: <https://github.com/tylervu-unlv/Group-1/pull/53>

```

===== test session starts =====
platform linux -- Python 3.10.12, pytest-8.4.2, pluggy-1.6.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /mnt/c/Users/administrators1/Group-1/tdd_lab
configfile: pytest.ini
plugins: cov-7.1.0
collected 9 items

tests/test_counter.py::TestCounterEndpoints::test_create_counter PASSED [ 11%]
tests/test_counter.py::TestCounterEndpoints::test_list_all_counters PASSED [ 22%]
tests/test_counter.py::TestCounterEndpoints::test_increment_counter PASSED [ 33%]
tests/test_counter.py::TestCounterEndpoints::test_non_existent_counter PASSED [ 44%]
tests/test_counter.py::TestCounterEndpoints::test_delete_counter PASSED [ 55%]
tests/test_counter.py::TestCounterEndpoints::test_delete_counter_not_found PASSED [ 66%]
tests/test_counter.py::TestCounterEndpoints::test_get_non_existent_counter PASSED [ 77%]
tests/test_counter.py::TestCounterEndpoints::test_invalid_method_on_counter FAILED [ 88%]
tests/test_counter.py::TestCounterEndpoints::test_create_invalid_counter_name PASSED [100%]

===== FAILURES =====
TestCounterEndpoints.test_invalid_method_on_counter

self = <tests.test_counter.TestCounterEndpoints object at 0x7635452c24a0>, client = <FlaskClient <Flask 'src.counter'>>

    def test_invalid_method_on_counter(self, client):
        """It should return 405 with a JSON error body when using an unsupported method"""
        client.post('/counters/baz')
        result = client.put('/counters/baz')
        assert result.status_code == status.HTTP_405_METHOD_NOT_ALLOWED
>       assert 200 == 405
E       + where 200 = <WrapperTestResponse streamed [200 OK]>.status_code
E       + and 405 = status.HTTP_405_METHOD_NOT_ALLOWED

tests/test_counter.py:107: AssertionError

===== tests coverage =====
coverage: platform linux, python 3.10.12-final-0

Name                Stmts   Miss  Cover   Missing
-----
src/__init__.py        3      0   100%
src/counter.py       37      1    97%    55
src/status.py         8      0   100%
TOTAL                 48      1    98%

===== short test summary info =====
FAILED tests/test_counter.py::TestCounterEndpoints::test_invalid_method_on_counter - assert 200 == 405
===== 1 failed, 8 passed in 0.59s =====

```

This failure isn't mine. `test_invalid_method_on_counter` assumed `PUT` would always be rejected, but a teammate's increment feature made `PUT` valid on that same route. Two correct features just clashed once merged. I flagged it to the team.

Retrieving an Existing Counter (Pool Item 2)

Issue: <https://github.com/tylervu-unlv/Group-1/issues/62>

The goal was simple: `GET /counters/<name>` should hand back a counter's current value if it exists. It reuses the `counter_exists()` helper that was already sitting in the worked example, so I didn't have to reinvent that check.

Pull request: <https://github.com/tylervu-unlv/Group-1/pull/63> Pending review

Red

I wrote `test_get_counter` before any implementation existed: post a counter so I know one's there, then `GET` it back and expect a 200 with the value. Running it against the unmodified code gave me exactly the failure I wanted:

```

FAILED tests/test_counter.py::TestCounterEndpoints::test_get_counter - assert 405 == 200

```

405 Method Not Allowed — because there simply wasn't a `GET` route on that path yet. Good, clean red.

Green

```

@app.route('/counters/<name>', methods=['GET'])
def get_counter(name):
    """Retrieve an existing counter"""
    if not counter_exists(name):

```

```
    return jsonify({"error": f"Counter {name} does not exist"}), status.HTTP_404_NOT_FOUND
    return jsonify({name: COUNTERS[name]}), status.HTTP_200_OK
```

```
tests/test_counter.py::TestCounterEndpoints::test_create_counter PASSED
tests/test_counter.py::TestCounterEndpoints::test_get_counter PASSED
2 passed in 0.56s
```

Refactor

Once it was green, I noticed `create_counter` and `get_counter` were both building their success response the exact same way, just with different status codes. Pulled that into one helper:

```
def counter_response(name, http_status):
    """Build the standard JSON response for a counter"""
    return jsonify({name: COUNTERS[name]}), http_status
```

Both routes now call it instead of repeating themselves, and the suite still passes after the cleanup:

```
tests/test_counter.py::TestCounterEndpoints::test_create_counter PASSED
tests/test_counter.py::TestCounterEndpoints::test_get_counter PASSED
2 passed in 0.54s
```

Reviewing a teammate's PR

PR #56 tylervu-unlv, rejecting non-alphanumeric counter names: <https://github.com/tylervu-unlv/Group-1/pull/56>

"Looks great!! Clean check, correctly placed before the duplicate check, and the status code addition to status.py is exactly right. Just one tiny typo: 'coutner' should be 'counter' in the test description comment."

Approved it after that.

PR: <https://github.com/tylervu-unlv/Group-1/pull/63>

As of turning this in, I haven't gotten feedback back on my own PR yet; the link's above once it comes through.

Phase 3 Continuous Integration

Getting the CI repo ready

My fork: <https://github.com/zombieg1rl/Group-1>

The CI setup PR was already merged into main by the time I got here, so syncing my fork pulled in `ci_lab/` and `.github/workflows/ci.yml` automatically. Ran the baseline suite to confirm everything worked before I touched anything:

```
python3 -m pytest --cov=src --cov-report=term-missing
```

22 tests passed, 93% coverage

Enhancement #6 CI visibility & docs

Issue: <https://github.com/tylervu-unlv/Group-1/issues/79>

I went with documentation and a status badge, mostly because I liked the idea of the README actually explaining itself instead of just being a wall of setup instructions. Added a live badge tied to the workflow:

```
[![CI
Workflow](https://github.com/tylervu-unlv/Group-1/actions/workflows/ci.yml/badge.svg)](https://github
.com/tylervu-unlv/Group-1/actions/workflows/ci.yml)
```

And wrote a short section explaining what the pipeline actually checks (the Python 3.9/3.10/3.11 matrix, the Flake8 lint pass) plus a walkthrough for anyone landing on a failed run: go to Actions, open the run, expand the failing step, read the output.

Pull request: <https://github.com/tylervu-unlv/Group-1/pull/80> **Pending review**

Breaking it on purpose

For the controlled failure, I dropped this into counter.py, sitting all by itself at the bottom of the file, outside any function:

```
print(this_variable_does_not_exist)
```

I actually expected this to trip the linter, not the tests. It didn't. Instead, the whole test suite failed to even start:

```
tests/test_counter.py:15: in <module>
    from src import app
src/__init__.py:9: in <module>
    from src.counter import app
src/counter.py:139: in <module>
    print(this_variable_does_not_exist)
E   NameError: name 'this_variable_does_not_exist' is not defined
ERROR tests/test_counter.py - NameError: name 'this_variable_does_not_exist' is not defined
Interrupted: 1 error during collection
Error: Process completed with exit code 2.
```

Why it broke where it did

Turns out this comes down to when Python actually runs code. A line sitting at the top level of a module runs the second that module gets imported — it doesn't wait for a function to be called. Since the test file imports the app straight from counter.py, my broken line fired before pytest could even finish collecting the tests, let alone run them. It's a more dramatic failure than a lint warning would've been, since it takes down the entire suite instead of flagging one style issue.

Fixing it and watching it go green again

Deleted the line, reran locally to make sure I hadn't broken anything else:

```
22 passed in 0.28s
```

Coverage held steady at 93%, which made sense since that broken line was never part of any tested path to begin with. Pushed the fix (commit 94abf7f), and the Actions run for PR #80 turned green across all three Python versions in the matrix.

Reviewing a teammate's PR

PR #86 VegasBear, coverage artifact upload: <https://github.com/tylervu-unlv/Group-1/pull/86>

I liked that they used matrix.python-version in the artifact name so each Python version gets its own separate report instead of overwriting one another, and if-no-files-found: error means the step actually fails loudly if the coverage report doesn't show up, instead of silently uploading nothing.

“Nice! Clean workflow changes and `matrix.python-version` is a smart touch.”

Same as Phase 2, I hadn't heard back on my own PR yet by the time I put this report together.

Reflection

A lot of the "best practices" I'd only ever heard about in lecture actually clicked doing this. Watching coverage numbers shift as I added tests made it obvious what that percentage is actually measuring, and reviewing my teammates' PRs taught me that a passing test isn't automatically a good test — plenty of room for something to pass while still missing an edge case.

The most interesting moment for me was probably the controlled failure in Phase 3. I fully expected it to fail at the linting step, and instead it failed earlier, at test collection, because the broken line was sitting at module level instead of inside a function. Chasing down why taught me more about how Python actually executes code than I expected to learn from a CI exercise — which was honestly useful, since Python isn't my strongest language.

Peer review pushed me to stop just saying "looks good" and actually back it up — pointing to specific lines, asking for evidence that wasn't there yet, reading code more carefully than I might have otherwise. Getting that same kind of feedback on my own PRs made it click why that matters. And honestly, the git hiccups, the merge conflicts, the random credential and

setup issues along the way ended up teaching me almost as much as the actual coding did. It's messier than solo homework, but it felt a lot closer to what real collaborative development is actually like.